

100

Aung Aung

Other

PDF generated at: 29 Jul 2026 06:03:50 UTC

[View this report on HackerRank](#)

Score

100% • 80.0 / 80
scored in CodePath TIP103: Unit 8 Assessment,
Version A - Summer 2026 in 32 min 16 sec on
28 Jul 2026 22:29:20 PDT

Integrity Issues

Medium

Integrity Summary

Detected code similarity with other submissions

Candidate Information

Email	aungag1998@gmail.com
Test	CodePath TIP103: Unit 8 Assessment, Version A - Summer 2026
Candidate Packet	View
Taken on	28 Jul 2026 22:29:20 PDT
Time taken	32 min 16 sec/ 90 min
Personal Member ID	142936
Email Address with CodePath	aungag1998@gmail.com
Github username with CodePath	jaz4Aung
Invited by	CodePath

Skill Distribution



There is no associated skills data that can be shown for this assessment

Tags Distribution



There is no associated tags data that can be shown for this assessment

Questions

Coding Questions • 60 / 60

No.	Question	Skill	Score	Code Quality	Time Taken	Status
1	Remove Kth Smallest Node Coding	-	20/20	-	5 min 51 sec	

2	Remove Node BST Coding	-	20/20 	-	6 min 57 sec	
3	Two Sum BST Coding	-	20/20	-	5 min 31 sec	

Multiple Choice + Debugging • 20 / 20

No.	Question	Skill	Score	Code Quality	Time Taken	Status
4	What is the time complexity of <code>sum_tree_levels()</code> ? Multiple Choice	-	5/5	-	2 min 43 sec	
5	What is the space complexity of <code>max_depth_sum()</code> ? Multiple Choice	-	5/5	-	43 sec	
6	What is the output of the following code snippet? Multiple Choice	-	5/5	-	48 sec	
7	Find the bug! Coding	-	5/5	-	9 min	

1. Remove Kth Smallest Node

 Correct

Coding

Question description

Given the `root` of a binary search tree (BST) and an integer `k`, write a function to return the `kth` smallest value (1-indexed) of all the values of the nodes in the tree.

Example 1:

Input: root = [3,1,4,None,2], k = 1

```
  3
 /\
1  4
 \
  2
```

Output: 1

Example 2:

Input: root = [5,3,6,2,4,None,None,1], k = 3

```
  5
 /\
 3 6
 /\
2  4
 /
1
```

Output: 3

Example 3:

Input: root = [2,1,3], k = 2

```
  2
 /\
1  3
```

Output: 2

Candidate's Solution

Language used: Python 3

```
1  #!/bin/python3
2
3  import math
4  import os
5  import random
6  import re
7  import sys
8  import ast
9
10 from typing import Optional, List
11
```

```
12 class TreeNode:
13     def __init__(self, val=0, left=None, right=None):
14         self.val = val
15         self.left = left
16         self.right = right
17
18
19 def kth_smallest(root, k):
20     # Write your code here
21     if not root:
22         return None
23     values = []
24     def dfs(current):
25
26         if not current:
27             return
28         values.append(current.val)
29
30         dfs(current.left)
31         dfs(current.right)
32
33     dfs(root)
34
35     values= sorted(values)
36
37     return values[k-1]
38 def build_tree(nodes, index=0):
39     if index < len(nodes) and nodes[index] is not None:
40         node = TreeNode(nodes[index])
41         node.left = build_tree(nodes, 2 * index + 1)
42         node.right = build_tree(nodes, 2 * index + 2)
43         return node
44     return None
45
46 if __name__ == '__main__':
47     outfile = open(os.environ['OUTPUT_PATH'], 'w')
48     input_data = sys.stdin.read().strip().splitlines()
49
50     for data in input_data:
51
52         tree_data, val = data.split('],')
53         tree_data += ']'
54
55         try:
56             tree_list = ast.literal_eval(tree_data)
57         except (SyntaxError, ValueError) as e:
```

```

58         print(f"Error parsing tree data: {e}")
59         continue
60
61     try:
62         k = int(val.strip())
63     except ValueError as e:
64         print(f"Error parsing value of k: {e}")
65         continue
66
67     root = build_tree(tree_list)
68     result = kth_smallest(root, k)
69
70     outfile.write(str(result) + '\n')
71 outfile.close()

```

TESTCASE	DIFFICULTY	TYPE	STATUS	SCORE	TIME TAKEN	MEMORY USED
Basic Case	Easy	Hidden	Success	0	0.036 sec	11.6 KB
Basic Case	Easy	Hidden	Success	0	0.0335 sec	11.7 KB
Tree with null nodes	Easy	Hidden	Success	0	0.0341 sec	11.7 KB
Single Node Tree	Easy	Hidden	Success	0	0.0321 sec	11.7 KB
Complete Binary Tree	Easy	Hidden	Success	0	0.0733 sec	11.7 KB
k = Number of Nodes	Easy	Hidden	Success	0	0.0316 sec	11.7 KB

Tree with Duplicates	Easy	Hidden	Success	0	0.0328 sec	11.7 KB
Large Tree	Easy	Hidden	Success	0	0.0304 sec	11.7 KB
Large k with Balanced Tree	Easy	Hidden	Success	0	0.0288 sec	11.7 KB
Pass/Fail Case	Easy	Hidden	Success	20	0.0294 sec	11.6 KB

🚫 No comments.

2. Remove Node BST

📝 Correct

Coding

Question description

Given the root of a binary search tree, remove the node with the value `val` into the tree. All nodes in the tree are guaranteed to be unique. Return the root of the modified tree.

If you need to replace a parent node with two children, use the in-order successor of that node. The in-order successor is the node with the smallest value greater than the value of the removed node.

Example 1:

Input: root = [5, 3, 6, 2, 4, None, 7], key = 3

Output: [5, 4, 6, 2, None, None, 7]

Example 2:

Input: root = [5, 3, 6, 2, 4, None, 7], key = 0

Output: [5, 3, 6, 2, 4, None, 7]

Example 3:

Input: root = [], key = 0

Output: []

Candidate's Solution

Language used: Python 3

```
1  #!/bin/python3
2
3  import math
4  import os
5  import random
6  import re
7  import sys
8  import ast
9  import json
10
11 class TreeNode:
12     def __init__(self, val=0, left=None, right=None):
13         self.val = val
14         self.left = left
15         self.right = right
16 # Enter your code here. Read input from STDIN. Print output to STDOUT
17 def delete_node(root, key):
18     # Write your code here
19
20     if not root:
21         return None
22
23     if key < root.val:
24         root.left = delete_node(root.left, key)
25
26     elif key > root.val:
27         root.right = delete_node(root.right, key)
28
29     else:
30         if not root.left:
31             return root.right
32
33         if not root.right:
34             return root.left
35
36
37         nextNode = root.right
38         while nextNode.left:
```

```
39         nextNode = nextNode.left
40
41         root.val = nextNode.val
42         root.right = delete_node(root.right, nextNode.val)
43
44     return root
45
46 def build_tree_from_list(lst):
47     if not lst:
48         return None
49
50     nodes = [TreeNode(val) if val is not None else None for val in lst]
51     kids = nodes[::-1]
52     root = kids.pop()
53
54     for node in nodes:
55         if node:
56             if kids: node.left = kids.pop()
57             if kids: node.right = kids.pop()
58
59     return root
60
61 def tree_to_list(root):
62     if not root:
63         return []
64
65     result, queue = [], [root]
66     while any(queue):
67         node = queue.pop(0)
68         if node:
69             result.append(node.val)
70             queue.append(node.left)
71             queue.append(node.right)
72         else:
73             result.append(None)
74
75     # Remove trailing None values
76     while result and result[-1] is None:
77         result.pop()
78
79     return result
80
81 if __name__ == '__main__':
82     input_data = sys.stdin.read().strip().splitlines()
83
84     for data in input_data:
```

```
85 tree_data, val = data.split(']',')
86 tree_data += ']'
87 val = int(val.strip())
88
89 tree_list = ast.literal_eval(tree_data)
90
91 root = build_tree_from_list(tree_list)
92 result = delete_node(root, val)
93
94 print(tree_to_list(result))
95
```

TESTCASE	DIFFICULTY	TYPE	STATUS	SCORE	TIME TAKEN	MEMORY USED
Basic Case	Easy	Hidden	Success	0	0.0298 sec	11.4 KB
Delete Leaf Node	Easy	Hidden	Success	0	0.0402 sec	11.4 KB
Delete Node with One Child	Easy	Hidden	Success	0	0.0366 sec	11.4 KB
Delete Node with Two Children	Easy	Hidden	Success	0	0.0345 sec	11.4 KB
Delete Root Node	Easy	Hidden	Success	0	0.0376 sec	11.4 KB
Delete Node Not Present in Tree	Easy	Hidden	Success	0	0.0324 sec	11.3 KB
Delete Node in an Empty Tree	Easy	Hidden	Success	0	0.0306 sec	11.4 KB

Delete Only Node in Tree	Easy	Hidden	Success	0	0.0304 sec	11.4 KB
Tree with All Nodes Having Same Value	Easy	Hidden	Success	0	0.0321 sec	11.4 KB
Delete Node with Deep Children	Easy	Hidden	Success	0	0.036 sec	11.4 KB
Left-Skewed Tree (Only Left Children)	Easy	Hidden	Success	0	0.0336 sec	11.3 KB
Right-Skewed Tree (Only Right Children)	Easy	Hidden	Success	0	0.0324 sec	11.4 KB
Balanced Tree	Easy	Hidden	Success	0	0.031 sec	11.3 KB
Pass/Fail Testcases	Easy	Hidden	Success	20	0.0398 sec	11.5 KB

 No comments.

3. Two Sum BST

 Correct

Coding

Question description

Given the `root` of a binary search tree and an integer `k`, return `True` if there exist two nodes in the BST such that the sum of their values is equal to `k`, and `False` otherwise.

Example 1:

Input: `root = [5,3,6,2,4,null,7]`, `k = 9`

```
  5
 / \
3   6
/\  \
2 4  7
```

Output: `True`

Explanation: $2 + 7 = 9$

Example 2:

Input: `root = [5,3,6,2,4,null,7]`, `k = 28`

```
  5
 / \
3   6
/\  \
2 4  7
```

Output: `False`

Explanation: There is no pair with sum equal to 28.

Example 3:

Input: `root = [2,1,3]`, `k = 1`

```
  2
 / \
1   3
```

Output: `False`

Explanation: There is no pair with sum equal to 1.

Candidate's Solution

Language used: **Python 3**

```
1 #!/bin/python3
2
3 import math
4 import os
5 import random
6 import re
7 import sys
8 import ast
```

```
9
10 from typing import List, Optional
11
12 class TreeNode:
13     def __init__(self, val=0, left=None, right=None):
14         self.val = val
15         self.left = left
16         self.right = right
17
18
19 def find_target(root: TreeNode, k: int) -> bool:
20     # Write your code here
21
22     seen = set()
23
24     def dfs(node):
25         if not node:
26             return False
27
28         if k - node.val in seen:
29             return True
30         seen.add(node.val)
31
32         return dfs(node.left) or dfs(node.right)
33
34     return dfs(root)
35
36 def list_to_tree_node(lst: List[Optional[int]]) -> Optional[TreeNode]:
37     if not lst:
38         return None
39
40     root = TreeNode(lst[0])
41     queue = [root]
42     i = 1
43
44     while i < len(lst):
45         node = queue.pop(0)
46         if i < len(lst) and lst[i] is not None:
47             node.left = TreeNode(lst[i])
48             queue.append(node.left)
49         i += 1
50         if i < len(lst) and lst[i] is not None:
51             node.right = TreeNode(lst[i])
52             queue.append(node.right)
53         i += 1
54
```

```

55     return root
56
57 if __name__ == '__main__':
58     outfile = open(os.environ['OUTPUT_PATH'], 'w')
59     input_data = sys.stdin.read().strip().splitlines()
60
61     for data in input_data:
62         tree_data, val = data.split('],')
63         tree_data += ']'
64         val = int(val.strip())
65
66         tree_list = ast.literal_eval(tree_data)
67
68         root = list_to_tree_node(tree_list)
69         result = find_target(root, val)
70
71         outfile.write(str(result) + '\n')
72     outfile.close()
73

```

TESTCASE	DIFFICULTY	TYPE	STATUS	SCORE	TIME TAKEN	MEMORY USED
Basic Case: True	Easy	Hidden	Success	0	0.04 sec	11.6 KB
Basic Case: False	Easy	Hidden	Success	0	0.0376 sec	11.6 KB
Tree with No Matching Pairs	Easy	Hidden	Success	0	0.0341 sec	11.6 KB
Tree with Single Node	Easy	Hidden	Success	0	0.0329 sec	11.6 KB
Tree with Duplicate Values	Easy	Hidden	Success	0	0.0464 sec	11.6 KB

Empty Tree	Easy	Hidden	Success	0	0.0333 sec	11.6 KB
Large Tree with Multiple Levels	Easy	Hidden	Success	0	0.0334 sec	11.6 KB
Tree with Large Numbers	Easy	Hidden	Success	0	0.0319 sec	11.6 KB
Tree with Negative Values	Easy	Hidden	Success	0	0.031 sec	11.6 KB
Tree with All Nodes the Same	Easy	Hidden	Success	0	0.0313 sec	11.6 KB
Pass/Fail Case	Easy	Hidden	Success	20	0.0316 sec	11.6 KB

🚫 No comments.

4. What is the time complexity of `sum_tree_levels()`?

✅ Correct

Multiple Choice

Question description

```
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.value = val
        self.left_child = left
        self.right_child = right

def sum_tree_levels(node):
```

```
def helper(n, depth, level_sums):
    if not n:
        return
    if depth == len(level_sums):
        level_sums.append(0)
    level_sums[depth] += n.value
    helper(n.left_child, depth + 1, level_sums)
    helper(n.right_child, depth + 1, level_sums)

level_sums = []
helper(node, 0, level_sums)
return level_sums
```

Candidate's Solution

Options: (Expected answer indicated with a tick)

☐ $O(1)$

☐ $O(\log n)$

☒ $O(n)$



☐ $O(n \log n)$

 No comments.

5. What is the space complexity of `max_depth_sum()`?

 Correct

Multiple Choice

Question description

```
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.value = val
        self.left_child = left
        self.right_child = right

def max_depth_sum(node):
    def depth_and_sum(n):
        if not n:
            return (0, 0)

        left_depth, left_sum = depth_and_sum(n.left_child)
        right_depth, right_sum = depth_and_sum(n.right_child)

        current_depth = max(left_depth, right_depth) + 1
        current_sum = left_sum + right_sum + n.value

        return (current_depth, current_sum)

    depth, total_sum = depth_and_sum(node)
    return total_sum
```

Candidate's Solution

Options: (Expected answer indicated with a tick)

☐ $O(1)$ ☐ $O(\log n)$

☒ $O(n)$ ☐ $O(n \log n)$  No comments.

6. What is the output of the following code snippet?

 Correct

Multiple Choice

Question description

```
from collections import deque

class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

def mystery_function(root):
    if not root:
        return []

    queue = deque([root])
    result = []

    while queue:
        node = queue.popleft()
        result.append(node.val * 2)

        if node.left:
            queue.append(node.left)
        if node.right:
            queue.append(node.right)
```

```
return result
```

```
# Test Cases
```

```
root = TreeNode(1)
root.left = TreeNode(2)
root.right = TreeNode(3)
root.left.left = TreeNode(4)
root.left.right = TreeNode(5)
root.right.left = TreeNode(6)
root.right.right = TreeNode(7)
```

```
print(mystery_function(root))
```

Candidate's Solution

Options: (Expected answer indicated with a tick)



<p>[2, 4, 6, 8, 10, 12, 14]<!-- notionvc: 3f8cc443-cddf-41d3-9cd7-783262be17bd --><!-- notionvc: 785744cc-6336-49ea-a10c-2d7671eb0ff9 --><!-- notionvc: 5b597bf0-b13c-4361-94af-1717d8005a35 --><!-- notionvc: 7359b583-b11a-4222-86a7-0c84ad67715e --></p>



<p>[1, 2, 3, 4, 5, 6, 7]<!-- notionvc: c522cffa-e4d1-45e2-8851-ff46ba1bce23 --><!-- notionvc: 89026cf7-d4bf-4b4a-869d-4868c011407b --><!-- notionvc: 0d0212d8-48df-42bf-9162-6bdd538be2be --><!-- notionvc: 55fdf1d2-3bb7-4b79-9255-b8612dd0850d --></p>



<p>[4, 2, 5, 1, 6, 3, 7]<!-- notionvc: 58b264a5-3428-480c-a05a-62fb0ee204d5 --><!-- notionvc: 0cd8e23a-d3cb-45c6-8a97-7588f6022f57 --><!-- notionvc: 4c910126-cb84-40a3-beac-6f75004ac2cb --><!-- notionvc: a5b43cc9-8ba4-46c3-9188-959602833bc2 --></p>



<p>[1, 3, 7, 6, 2, 5, 4]<!-- notionvc: 2a3ed9eb-ff3e-49e4-a0c5-6e433d398ee1 --><!-- notionvc: 82a1d053-0776-4dcb-b112-fe392deeb83d --><!-- notionvc: eb4cecede-a6be-4620-

b260-e66a6104e69d --><!-- notionvc: 4094b50d-e35d-459b-8918-c7bc671ac37f --></p>

⚠ No comments.

7. Find the bug!

✓ Correct

Coding

Question description

The function below incorrectly implements `postorder_traversal()`. Implemented correctly, `postorder_traversal()` should accept the `root` of a binary tree and returns an array representing the postorder traversal of its nodes' values.

Identify any bug(s) within the given implementation and correct the code so that it successfully passes the provided test cases.

Candidate's Solution

Language used: Python 3

```
1  #!/bin/python3
2
3  import math
4  import os
5  import random
6  import re
7  import sys
8  import ast
9
10 class TreeNode:
11     def __init__(self, val=0, left=None, right=None):
12         self.val = val
13         self.left = left
14         self.right = right
15
16 def postorder_traversal(root):
17     if not root:
18         return []
19     result = []
```

```
20     result.extend(postorder_traversal(root.left))
21     result.extend(postorder_traversal(root.right))
22     result.append(root.val)
23
24     return result
25
26 def list_to_bst(lst, i=0):
27     if lst is None or i >= len(lst) or lst[i] is None:
28         return None
29
30     root = TreeNode(lst[i])
31     root.left = list_to_bst(lst, 2 * i + 1)
32     root.right = list_to_bst(lst, 2 * i + 2)
33
34     return root
35
36 def bst_to_list(root):
37     if not root:
38         return []
39
40     result = []
41     queue = [root]
42
43     while queue:
44         node = queue.pop(0)
45         if node:
46             result.append(node.val)
47             queue.append(node.left)
48             queue.append(node.right)
49         else:
50             result.append(None)
51
52     while result and result[-1] is None:
53         result.pop()
54
55     return result
56
57 if __name__ == '__main__':
58     input_data = sys.stdin.read().strip()
59     input_list = ast.literal_eval(input_data)
60
61     root = list_to_bst(input_list)
62     result = postorder_traversal(root)
63     print(result)
```

TESTCASE	DIFFICULTY	TYPE	STATUS	SCORE	TIME TAKEN	MEMORY USED
Pass/Fail Case	Easy	Hidden	Success	5	0.0344 sec	10.9 KB

 No comments.